Coding Challenge + Poster Presentation (worth 40% of the total module assessment)

Weighting summary

	Criterion	Weight
1	Data cleaning and preprocessing	**10%**
2	Comparison of sequential and parallel execution	**15%**
3	Data parallelism	**10%**
4	Task parallelism (any 2 of: feature extraction, gradient computation, hyperparameter search)	**20%**
5	Identifying non-determinism (any 2) and fixes	**15%**
6	Well-formatted, reproducible notebook	**10%**
7	Poster presentation of the solution	**20%**
	Total	**100%**

- Results claimed on the poster must be traceable to the notebook; unsupported claims are scored at the lower level.

Detailed rubric

Criterion and evidence expected	Weight	Excellent (90–100%)	Good (70–89%)	Satisfactory (50–69%)	Needs improvement (0–49%)
1. Data cleaning and preprocessing *Loads the Parquet files efficiently and cleans them before any modelling.* • Column pruning / filters or footer inspection used when reading • Handles meter errors (e.g. 269,097-mile trip), negative fares, zero-distance and zero/negative duration trips • Removes Dec/Feb spill-over pickups • Treats Flex Fare nulls deliberately; encodes zone and rate codes as categories • Avoids leakage; time-based train/test split	**10%**	All issues found and handled with stated, justified thresholds; before/after row counts reported; leakage and time-split handled correctly; efficient Parquet reading.	Most issues handled with reasonable thresholds; minor gaps in justification or one missed issue; split and leakage handled.	Basic cleaning only (e.g. drops nulls, removes some outliers); thresholds arbitrary; random split or minor leakage.	Little or no cleaning; obvious errors remain; leakage present; whole file loaded without thought.
2. Comparison of sequential and parallel execution *Runs the same workload sequentially and in parallel and analyses the difference.* • Identical work and outputs in both versions (results checked to match) • Wall-clock timing with repeats (e.g. mean of 3+ runs); warm-up considered • Speedup and efficiency for different worker counts (e.g. 2, 4) • Chart of speedup vs workers • Discussion using Amdahl's law: estimated parallel fraction p and overheads (communication, process start-up, data transfer, load imbalance, I/O)	**15%**	Fair, repeated benchmark across several worker counts; outputs verified equal; clear chart; speedup explained with Amdahl's law and measured overheads.	Sound benchmark with chart; speedup reported; some explanation of why it is below ideal.	Single timing for each version; speedup stated but little analysis; unclear whether outputs match.	No valid comparison, or versions do different work; no timings or wrong conclusions.
3. Data parallelism *Splits the data into shards and applies the same computation to each shard in parallel, then combines results correctly.* • Sharding scheme chosen and justified • Same function applied to every shard • Correct combination of partial results (partial sums and counts, concatenation in a fixed order) • Result checked against the sequential result;	**10%**	Well-justified sharding; balanced shards; correct, verified merge of partial results; scales with workers; memory use considered (e.g. iter_batches).	Working data-parallel pipeline with sensible shards and correct merge; limited justification or verification.	Parallel code runs but shards are unbalanced, merge is partly wrong, or results not verified.	No real data parallelism (e.g. a single worker), or results incorrect.

Criterion and evidence expected	Weight	Excellent (90–100%)	Good (70–89%)	Satisfactory (50–69%)	Needs improvement (0–49%)
4. Task parallelism (any 2 of: feature extraction, gradient computation, hyperparameter search) *Implements two task-parallel patterns, each running different, independent work concurrently* • Feature extraction: independent feature tasks run concurrently and recombined in a fixed column order • Gradient computation: gradients computed per shard/worker and averaged (all-reduce style) to update one shared model; • Hyperparameter search: independent trials (grid or random, optionally early stopping) run as separate jobs; best configuration chosen on validation data, not test • Each part explains why the work is independent and reports timing	**20%**	Both patterns correct, clearly independent tasks, verified results, timing reported, and load imbalance between tasks discussed.	Both patterns implemented and working; limited verification or discussion.	One pattern done well, or both done with errors (e.g. tasks secretly sequential, results combined in wrong order).	Neither pattern works, or the work shown is not task parallelism.
5. Identifying non-determinism (any 2) and fixes *Finds two real sources of run-to-run variation in the parallel solution, shows them, and fixes them.* • Examples: unseeded randomness (splits, sampling, model init, random search); results collected in completion order (as_completed / next free shard); floating-point sums combined in a different order; dataset shuffling; set/dict or hash ordering; thread-count dependent library maths • Evidence: two or more runs showing the differing output • Fixes: fixed seeds per worker, sorting results by shard/task ID, deterministic reduction order, fixed shard-to-worker mapping, pinned thread counts and library versions • Re-run after the fix showing identical output	**15%**	Two genuine sources identified with evidence from repeated runs; each fix explained and shown to make the output identical; limits of the fix discussed.	Two sources identified and fixed; evidence or re-run check partly missing.	Only one source handled properly, or sources named without evidence or working fixes.	Non-determinism not identified, or examples incorrect.

Criterion and evidence expected	Weight	Excellent (90–100%)	Good (70–89%)	Satisfactory (50–69%)	Needs improvement (0–49%)
6. Well-formatted, reproducible notebook *A clean notebook that another person can run and get the same results.* • Runs end to end after 'Restart and run all' without errors • Environment recorded (Python and library versions, requirements file); data source and file names stated • Seeds and configuration in one place; • Clear section headings and markdown explanations; outputs kept but not cluttered • Run time and hardware (cores/GPU) noted	**10%**	Runs cleanly first time; fully documented environment; clear structure and narrative; reusable functions; results match the poster.	Runs with minor fixes; mostly documented and well organised.	Needs several fixes to run; sparse explanation; messy or repeated code.	Does not run, or is disorganised with no explanation.
7. Poster presentation of the solution *A poster that communicates the problem, the parallel design and the results, and is defended in the presentation.* • Problem and dataset clearly stated • Pipeline diagram showing where data and task parallelism are used • Speedup chart and key results • Non-determinism findings and fixes summarised • Readable layout, labelled figures, concise text;	**20%**	Clear, visually strong poster that tells the full story; accurate figures; confident, correct answers to questions.	Clear poster covering most elements; minor layout or explanation gaps; answers mostly correct.	Poster incomplete or text-heavy; figures hard to read; weak answers.	Poster missing key parts or inaccurate; unable to explain the work.